UNX511: UNIX Systems Programming

Week 7-Session2

Lecturer: Shahdad Shariatmadari

Winter 2026

Agenda

- Review
- Some hints about assignment1
- Play Kahoot game
- Midterm exam format

What is UNIX fundamentally built around?

- Everything is a file
- Processes don’t share memory
- Kernel mediates all access
- File descriptors abstract resources
- Signals enable async communication

```
User Program
    ↓
System Call
    ↓
Kernel
    ↓
Resource (File / Process / Device)
```

Shell & File System

↓

File Programming

↓

fcntl / ioctl

↓

Processes & Signals

↓

IPC (coming next)

Week1,2

- Review UNIX file system
  - Basic commands
  - Permissions
  - File descriptors (0,1,2)
- Processes & Signals
- File Programming
  - open(), ready(), write()
- C Programming
  - MakeFile
  - Static/Dynamic Library
- Lab1

Week3

- Focused on
  - Working with processes
    - /proc, /proc/[pid]/status
    - VmRSS vs VmSize
  - File I/O
    - Buffered/unbuffered
    - seek(), stat(), st_mode structure
- Lab2

Week4

- Focused on
  - Used Socket : socket(AF_INET, SOCK_DGRAM, 0)
  - Used ioctl() : extract device spec
  - Core Idea:
    - User space → system call → kernel → device/network
  - dup()
- Lab3
  - Used socket/ioctl to extract network interface info: IPv4, MAC, …

Week5

- Focused on
  - Device driver
  - Character device drivers
- Lab4
  - implemented read()
  - Implemented ioctl()
  - Controlled kernel thread behavior
  - Used HALT / RESUME commands

Week6

- Focused on
  - fork() and signal()/sigaction()
  - Working with parent/child process
  - Change behavior of SIGUSR1, SIGINT, …
- Lab5
  - Coordinated parent/child
  - Used SIGUSR1 / SIGUSR2
  - Discarded Ctrl-C / Ctrl-Z
  - Used wait()

FD, Proc, Signal

- Each process:
  - has file descriptors
    - FD table
    - Points to kernel open-file table, Which points to inode
  - inherits FDs
  - can fork
  - can send signals
  - must handle termination
  - communicates with kernel via system calls

What happens after **fork()** to file descriptors?

- When you call `fork()`, the child process inherits **copies** of the parent's file descriptor (FD) table.
  - **Shared File Offset:** Since both the parent and child point to the same open file description, they share a single **read/write offset**. If the child reads 10 bytes, the parent's next read will start at byte 11.
  - **Independent FD Tables:** Although they share the same underlying file structures, the file descriptor integers themselves are in independent tables. Closing an FD in the child **does not** close it in the parent, and vice versa.
  - **Shared Status Flags:** Changes made via `fcntl()` to file status flags (like O_NONBLOCK or O_ASYNC) affect both processes.
  - **Reference Counting:** The kernel maintains a reference count for each open file description. The file is only truly closed by the OS once **all** processes holding a descriptor for it (parent, child, and any others) have closed

How to Approach Assignment 1

- **1) Build it in 5 milestones (don’t try everything at once)**
  - **Milestone A — File generator works**
    - Program asks for 3 sizes (MB)
    - Creates worker1.bin, worker2.bin, worker3.bin
  - **Milestone B — One worker works (no fork yet)**
    - Call xx_worker_process("worker1.bin", getpid()) directly
    - Verify it reads the file in 4KB chunks and prints progress

How to Approach Assignment 1

- **Milestone C — Fork 3 workers**
  - Parent forks 3 children
  - Each child calls xx_worker_process(fileX, parent_pid)
  - Parent waits for all children (waitpid loop)
- **Milestone D — Signals + sigaction in parent**
  - Parent handles **SIGUSR1 (warning)** and **SIGUSR2 (done)**
  - Parent prints and logs each event with the worker PID (from siginfo_t)
- **Milestone E — Secure logging**
  - Parent writes all events to syslog.log
  - Uses fcntl() write-lock before writing

Assignment 1: what your program must look like

- Your final solution should have these components:
- **Parent process**
  - Generates files (Part 1)
  - Forks 3 workers (Part 2)
  - Installs signal handlers using sigaction() (Part 3)
  - Logs events using xx_log_event() (Part 4)
  - Waits for all children to exit cleanly
- **Worker process**
  - Reads file in **4KB chunks**
  - Checks memory usage (/proc/self/status → VmRSS)
  - Sends **SIGUSR1** if memory exceeds 50MB
  - Sends **SIGUSR2** when finished reading file
  - Exits

Points to consider:

- Part 3 “signal()” :
  - The given template uses signal(), however it is asked for sigaction and siginfo_t:
    - Use sigaction(SIGUSR1, ...) and set sa_flags = SA_SIGINFO , handler(int sig, siginfo_t *info, void *ucontext), Then use info->si_pid to log which worker triggered it
- SIGUSR1 may never trigger. As mentioned in **Requirement:** To demonstrate SIGUSR1, each worker must intentionally increase memory usage enough to exceed 50MB.
  - You may produce two approach for demo:
    - Approach A (simple): malloc(60MB) once at start, keep pointer until exit
    - Approach B (realistic): store chunks into a growing buffer (harder)

Midterm

- Monday March 2, 2026
- Format is the same as previous quizzes
  - Part1: Theory
  - Part2: Practical